Thomas Nguyen
11888002
Professor Zhe Dang
Due 2/18/2026

---

1. (easy) Write psuedo-code for partition(A, p, q).

```
pivot = A[q] // highest element
i = p - 1 // smallest element

for j = (p to q - 1) do
    if A[j] <= pivot
    i++
    swap A[i] w/ A[j]

swap A[i+1] w/ A[q] // places pivot in correct pos
return i + 1
```

1. (standard) Consider insertsort. Suppose that the input array A has 1% probability to be monotonically decreasing. Show that, in this case, the average-case complexity of insertsort is Θ(n2).

Worst case of insertsort = Θ(n2)
Avg case of insertsort = Θ(n2)
Let

$$T_w(n) \text{ be worst case be of insertsort}$$

$$T_{avg}(n) \text{ be average time complexity of insertsort}$$

$$T(n) = T_w(n) + \frac{99}{100} T_{avg}(n)$$

$$T(n) = \frac{1}{100} * \Theta(n^2) + \frac{99}{100} * \Theta(n^2)$$

$$T(n) = (\frac{1 + 99}{100})\Theta(n^2)$$

$$T(n) = \Theta(n^2)$$

3. (not hard) Let iqsort(A, 1, n) be an algorithm that sorts an array A with n integers. It works as follows:

```
iqsort(A, p, q){
if p ≥ q, return;
r=partition(A, p, q);
//run quick sort on the low part
quicksort(A, p, r − 1);
//run insert sort on the high part
insertsort(A, r + 1, q);
}
```

Compute the best-case, worst-case, and average-case complexities of iqsort.

Partition:

LOW	|	HIGH
r-1	r	n-r

Run Quicksort on LOW
Run Insert Sort on HIGH

Let $T_{best}(n)$ be best case for iqsort

The best cases for

- Quicksort side: $O(n \log n)$
- Insertion sort side: $O(n^2)$
  So,

$$T_{best}(n) = O(n \log n) + O(n^2)$$

  Dominant term:

$$T(n) = \Theta(n^2)$$

Let T_W be worstcase for iq sort
If one side has size $n - 1$,

- Either quicksort runs on $n - 1$
- Or insertion sort runs on $n - 1$
  Both give:

$$T(n) = \Theta(n^2)$$

Let $T_{avg}(n)$ be average time case for iqsort

$$T_{avg}(n) = \frac{1}{n}\sum_{r=1}^{n}\{T_{avg}^{quick}(r-1) + T_{avg}^{insert}(n-r)\} + O(n)$$

$$= \frac{1}{n}\sum_{r=1}^{n}\{O(r-1)\log(r-1) + O(n-r)^2\} + O(n)$$

$$<= \frac{1}{n}\sum_{r=1}^{n}\{a(r-1)\log(r-1) + a(n-r)^2\} + a(n)$$

$$= \frac{a}{n}\sum_{r=1}^{n}(r-1)\log(r-1) + \frac{a}{n}\sum_{r=1}^{n}(n-r)^2 + a(n)$$

$$= \frac{a}{n}\sum_{r=1}^{n}(r-1)\log(r-1) + \frac{a}{n}\sum_{r=1}^{n}(r-1)^2 + a(n)$$

$$<= \frac{2a}{n}\sum_{r=1}^{n}(r-1)^2 + an$$

$$<= \frac{2a}{n}\int_{0}^{n} x^2\,dx + an$$

$$= \frac{2a}{n} * \frac{1}{3}n^3 + an$$

$$= O(n^2)$$

4. hard) Let mixsort(A, 1, n) be an algorithm that sorts an array A with n integers. It works as follows:
   mixsort(A, p, q){
   if p ≥ q, return;
   r=partition(A, p, q);
   //run mixsort on the low part
   mixsort(A, p, r − 1);
   //run insert sort on the high part
   insertsort(A, r + 1, q);
   }
   Compute the best-case, worst-case, and average-case complexities of mixsort.

Partition:

LOW	|	HIGH
r-1	r	n-r

Run mixSort on LOW
Run Insert Sort on HIGH

Let $T_{best}(n)$ be best time case for mix sort
insertion sort dominates which is

$$T_{best} = O(n^2)$$

Let $T_W(n)$ be worst time case for mixsort
reached with the most minimal element which is insertion sort which is

$$T_w(n) = O(n^2)$$

Let $T_{avg}(n)$ be average time case for mix sort

Guessing $T_{avg} = O(n^2) <= Cn^2$ for some c > 0

$$T_{avg}(n) = \frac{1}{n}\sum_{r=1}^{n}\{T_{avg}^{quick}(r-1) + a(n-r)\} + a(n)$$

Using I.H

$$\forall i < n, T_{avg} <= c * i^2$$

$$= \frac{1}{n}\sum_{r=1}^{n}\{c(r-1)^2 + a(n-r)^2\} + O(n)$$

$$<= \frac{1}{n}\sum_{r=1}^{n}\{c(r-1)^2 + \frac{a}{n}\sum_{r=1}^{n}(n-r)^2 + a(n)$$

$$= \frac{c+a}{n}\sum_{r=1}^{n}(r-1)^2 + an$$

$$<= \frac{c+a}{n}\int_{0}^{n} x^2\,dx + an$$

$$= \frac{c+a}{n} * \frac{1}{3}n^3 + an <= cn^2 \text{ when c>>a}$$

$$= O(n^2)$$